

ENS 491/492 – Graduation Project

Draft Final Report

Project Title:

Implementing Deep Neural Networks (DNNs) Using GPUs

Group Members:

Ada Boran Yılmaz (30789)

Enes Çağlar (31109)

Supervisor(s):

Ömer Ceylan

Date:

21/12/2025



TABLE OF CONTENTS

1. EXECUTIVE SUMMARY.....	3
2. PROBLEM STATEMENT.....	4
2.1. Objectives/Tasks.....	5
2.2. Realistic Constraints.....	6
3. METHODOLOGY.....	7
3.1. Optimization Methodology for EfficientNet-B0.....	7
3.1.1. Environment Consolidation.....	7
3.1.2. Pipelining in C++.....	8
3.1.3. Profiling.....	8
3.1.4. Programmatic Graph Surgery.....	8
3.1.5. Hand-Tuned CUDA Kernel Fusion.....	9
3.1.6. Verification and Bandwidth Saturation Metrics.....	9
3.2. Optimization Methodology for MobileNet.....	9
3.2.1. Baseline Framework and Engine-Level Optimization.....	10
3.2.2. The Process of Finding Bottlenecks in the MobileNet.....	10
3.2.3. Designing the CUDA Custom Kernel for Depthwise Convolutions.....	10
3.2.4. Custom CUDA Kernel Creation for Pointwise Convolutions.....	10
3.2.5. Validation and Performance Measurement.....	11
4. RESULTS & DISCUSSION.....	11
4.1. Results and Discussion for EfficientNet-B0.....	11
4.1.1. GPU Inference Performance and Batch Size Scaling for EfficientNet.....	11
4.1.2. The Framework Transition and Environmental Orchestration.....	12
4.1.3. Evaluation of System-Level Optimizations: C++ vs PyTorch.....	12
4.1.4. Layer-Level Bottleneck Analysis and the “Memory Wall”.....	13
4.1.5. Custom Fused CUDA Kernels and Bandwidth Saturation.....	13
4.1.6. Final Benchmarking.....	14
4.1.7. Evaluation Against Initial Objectives.....	14
4.2. Results and Discussion for MobileNet.....	15
4.2.1. CPU vs GPU Inference Performance for MobileNet.....	15
4.2.2. Comparison of PyTorch, ONNX Runtime vs TensorRT.....	16
4.2.3. FP32 vs FP16 Engines and Batch Size Scaling.....	16
4.2.4. Layer-Level Bottleneck Analysis using NVIDIA Nsight Systems.....	17
4.2.5. Custom CUDA Kernels vs. PyTorch Implementations.....	17
4.2.6. Evaluation using Multiple GPUs in A100, L4, and T4.....	18
4.2.7. Evaluation Against Initial Objectives.....	18
5. IMPACT.....	19
6. ETHICAL ISSUES.....	20
7. PROJECT MANAGEMENT.....	21
8. CONCLUSION AND FUTURE WORK.....	23
9. APPENDIX.....	24
9.1. EfficientNet-B0 Results.....	24
9.2. MobileNet Results.....	27
10. REFERENCES.....	34

1. EXECUTIVE SUMMARY

This project targets the fundamental performance gap of deploying DNNs on resource-constrained edge devices by focusing on overcoming the “Memory Wall”, where computational speed is outpacing memory bandwidth. We focus on EfficientNet-B0 and MobileNet, attempting to achieve physical limits using the GPU by bypassing standard software abstractions and relying on hand-tuned CUDA kernels.

We refactored this project in three levels of technical refinement. First, we ported inference logic from Python/PyTorch to a C++ environment, including system-level optimizations such as Asynchronous CUDA Streams and Pinned Memory to eliminate data transfer overhead. Second, we optimized the models into high-performance engines using the NVIDIA TensorRT. Finally, since standard libraries leave certain layers memory-bound, we did “graph surgery” and wrote custom kernels. In the case of EfficientNet-B0, we utilize ONNX Graph Surgeon to replace fragmented Swish-activations with fused CUDA kernels. For MobileNet, we consider Depthwise and Pointwise convolutions and develop specialized FP16 kernels, which use register-level accumulation and loop unrolling to minimize global memory access.

These were then experimentally validated on the NVIDIA TRUBA clusters: Tesla V100, P100, and A100. EfficientNet-B0 benchmarks on the P100 had a 67% increase in throughput, from 560 to 936 FPS, with the custom Swish kernel saturating 92.2% of physical bandwidth at 675 GB/s. Similarly, the optimizations of MobileNet on A100 achieved 2.07x latency reduction for Depthwise convolutions and a 2.61x speedup for Pointwise kernels over the PyTorch FP16 baseline. These results indicate that while automated engines provide a strong foundation, surgical kernel-level intervention is crucial to achieve maximum hardware utilization and real-time efficiency in deep learning deployment.

2. PROBLEM STATEMENT

With the growing adoption of artificial intelligence within actual applications, ranging from self-navigating systems to smarter monitoring, the need for efficiency in inference has dominated the deep learning landscape. Although models developed, like EfficientNet-B0 and MobileNet, tend to be more lightweight, the performance within the context of the edge-optimized GPUs has not reached the theoretical boundaries. The problem addressed within this research project revolves around the “Memory Wall”, where the enormous computational bandwidth available within the computational unit, the GPU, is impeded by the relatively slower data transfer across the global memory and the streaming multiprocessors.

Industry-grade libraries, including NVIDIA TensorRT, have been proven to perform strong optimization through layer fusion, precision, and adjustment. Most of these libraries assume every neural network consists of generic operations, without often optimizing unusual patterns found in activation functions, such as Swish, heavily utilized in the EfficientNet models. When implemented traditionally, all these operations can be broken down into several memory-heavy passes, causing the hardware to remain well below its rated bandwidth. What drives our idea is the need to move away from all high-level software frameworks. Through the use of custom-made kernels on the CUDA platform, we wanted to demonstrate the possibility of getting more performance through manually optimized software compared to compiler-optimized models.

Regarding the MobileNet model, the bottleneck problem is based on the optimization of depthwise separable convolution to prefer fewer parameters as well as fewer arithmetic operations when in opposition to the model’s optimization from the memory access pattern perspective in the GPU. The MobileNet model’s depthwise convolution operates on per-channel data with fewer arithmetic operations, thus this is memory-parallel, yet this entire process is inefficient concerning general support in GPU. The actual matter is that due to this ratio being inefficient in regards to the cost of arithmetic operation to memory access in the optimization problem, this model is lightweight yet composed of convolutional layers in depthwise as well as pointwise layers which indeed act as bottlenecks in performance. The problem is solved in this project through specializing performance bottleneck layers in an attempt to optimize using CUDA.

2.1. Objectives/Tasks

The project progressed according to a technical agenda to shift from the flexible development environment to the high-performance environment for the production runtime:

1. **Baseline Establishment:** Develop a benchmarking harness in PyTorch to evaluate the throughput (FPS) and latency of both EfficientNet-B0 and MobileNet on TRUBA's Tesla P100/V100 nodes.
 - Intended Result: To provide a measurable boundary beyond which the best frameworks fail.
2. **C++ Inference Engine Development:** Reimplementing the Inference Flow using the TensorRT API to avoid the Global Interpreter Locking (GIL) and Python-related bottlenecks.
 - Intended Result: A runtime system with low overhead that can support asynchronous execution and pinned memory management.
3. **Graph Surgery & Layer Fusion:** Leverage the Graph Surgeon library from ONNX for replacing inefficient Sigmoid-Multiply patterns with custom plugin layers.
 - Intended Result: An optimized model graph, which will inevitably require the inference engine to use specialized kernels.
4. **Custom CUDA Kernel Implementation:** Write and compile a combined Swish activation kernel in CUDA C++ that reduces memory access.
 - Intended Result: The execution speed should approach >90% of the physical memory bandwidth of the GPU that is to be used in this project.
5. **MobileNet Layer-wise Optimization and Hardware-Targeted Evaluation:** Optimize the CUDA kernels for the depthwise and pointwise layers in the MobileNet model to enhance memory access efficiency and arithmetic intensity on the GPU.
 - Intended Result: To improve the latency metric and show a substantial increase in the throughput compared to the PyTorch FP16 baseline models, as

a verification of the efficacy of the GPU-level optimizations for layers of various numbers of channels and resolutions.

2.2. Realistic Constraints

There are a number of real-world factors that influenced the technical approach of the project as follows:

- **Economic & Sustainability Constraints:** To edge-enabled development, the optimization of the utilization of available hardware resources is essential. Even through the optimization of the software that runs 67% faster compared to the previous generation Tesla P100 GPUs, we show that there is scope for sustainable AI that could consume reduced amounts of energy per inference.
- **Environmental & Resource Constraints:** This particular project was based on TRUBA High-Performance Computing resources. This added a “Permission Denied” constraint wherein it was prohibited, based on security policies, to use Nsight Compute for profiling at a hardware level. This was fixed by creating a benchmark test program using C++ wherein memory bandwidth efficiency was calculated mathematically based on execution times.
- **Technical & Compatibility Standards:** To make sure that our project is relevant to the industry, we followed a standard known as ONNX or Open Neural Network Exchange. We could use ONNX to remain compatible with other frameworks for training models and our own inferencing engines.
- **Manufacturability & Portability:** In the HPC context (library mixing), one of the conflicts encountered by Environment was tackled by using the Apptainer (formerly Singularity) “container” to develop a portable and bit-for-bit equivalent Environment with defined versions of CUDA, TensorRT, and C++ compilers.
- **Mobile and Edge Hardware Limitations:** MobileNet is designed to be deployed on edge and mobile platforms with hardware limitations, similar to IoT devices. These platforms require strong hardware constraints on memory, energy, and heat budgets. It

is less convenient to apply optimizations related to large batch sizes, very parallel computation code paths, and high precision computation on these platforms. Optimizations must, on the contrary, emphasize low latency and single batch processing with lightweight memory-efficient computation kernels, and not processing performance. The project depicts hardware limitations using single batch settings while optimizing the MobileNet kernel and focuses on the optimization of Depthwise and Pointwise convolutions with lower memory and kernel launching costs.

3. METHODOLOGY

The optimizations of EfficientNet-B0 and MobileNet were conducted through a multi-stage approach that transitioned from identifying high-level software bottlenecks to implementing low-level hardware solutions. The core objective was to move past the “Black Box” nature of automated optimization tools and manually enforce hardware-efficient execution patterns.

3.1. Optimization Methodology for EfficientNet-B0

3.1.1. Environment Consolidation

The first part of the methodology was to provide a stable development platform on the TRUBA High-Performance Computing (HPC) cluster. These early tests were plagued by “DLL Hell”, which arose due to inconsistencies between the host-level CUDA libraries and the use of Python-managed libraries, leading to Error 35, or failed CUDA initialization. To eliminate these problems, we employed the use of Apptainer, which allowed us to build a bit-for-bit reproducible platform. This provided us with the correct versions of the NVIDIA TensorRT 8.6, CUDA 12.0, and the GCC compiler, which allowed us to migrate from unstable Python code to high-performance C++ code.

3.1.2. Pipelining in C++

To reduce the Python Global Interpreter Lock (GIL) overhead, we reimplemented the inference engine in C++. The most significant methodological shift during this particular task was the introduction of Pinned (Page-Locked) Memory. Rather than using traditional CPU memory for this particular task, we employed `cudaMallocHost`, which ensured that the operating system did not move the data to disk storage. This particular implementation made it possible for the GPU device operation to leverage its capabilities of Direct Memory Access and ensured that the data pipe remained at par with the efficient execution of the EfficientNet-B0 backbone network. Further optimization for this particular task came from Asynchronous Streams of `cudaStream_t`, which ensured that the CPU computed the next buffer full of images while the execution was still pending on the finishing of the current inference task on the GPU device.

3.1.3. Profiling

However, before implementing our custom code, it was necessary to understand where the latency occurred within the hardware. The `trtexec` profile command with the `--exportProfile` flag helped analyze the timing profile for each layer of the EfficientNet-B0 model. From this, it emerged that, although the convolution layers were optimized, the Swish activation functions ($x * \text{sigmoid}(x)$) were the big bottlenecks. In the original graph, TensorRT was able to interpret this as a combination of Sigmoid and Multiply operations. Effectively, this required the GPU to access the same data twice, writing the result to the slower Global Memory, thus creating a big “Memory Wall” bottleneck.

3.1.4. Programmatic Graph Surgery

For resolving the issue of fragmentation in the activation function of the Swish network, we used the ONNX Graph Surgeon. We could not have worked on the huge activation file directly. We had to resort to writing a Python script for navigating through the activation computation graph. The step-by-step procedure followed:

- Searching for a particular topological structure: Sigmoid Node followed by a Mul Node with the same input.
- Removing these nodes from the graph.
- The use of the Swish Plugin node in their place.

This surgery changed the model from a general ONNX file to a special one that required the TensorRT engine to scan for a C++ implementation of our model.

3.1.5. Hand-Tuned CUDA Kernel Fusion

The smallest scale in the methodology was the creation of the custom CUDA kernel. We removed the intermediate writes in DRAM in the baseline using Kernel Fusion. Each GPU thread in the custom kernel was programmed to do the following:

- Load a single pixel value, denoted by 'x', into a high-speed Register.
- Calculate the Sigmoid result with the `__expf` fast-math intrinsic.
- The final multiplication ($x \cdot \text{Sigmoid}$) must be done inside the compute core.
- Write only the final result back to memory.

This lowered the number of memory transactions per pixel from about six to two (read and write).

3.1.6. Verification and Bandwidth Saturation Metrics

Since hardware performance counter support was prohibited in TRUBA due to some security issues in their `ERN_NVGPUCTRPERM` environment, we could not utilize them. We overcame this hurdle by creating our C++ benchmarking framework through which we calculated our Effective Bandwidth. This was achieved as we measured our kernel run time and calculated our efficiency as our run time is matched with Total Data Moved (Input + Output). This helped verify our claim that our code achieved speeds of 675 GB/s and thereby utilized 92.2% of Tesla P100's memory limit as our kernel code is near-perfectly optimized.

3.2. Optimization Methodology for MobileNet

The strategy used in the optimization of MobileNet followed a layer-centric methodology, since the main rationale behind this was that the MobileNet model's performance is largely affected by depthwise and pointwise separable convolutions. The depthwise and pointwise separable layers actively decrease the arithmetic intensity but increase the arithmetic overhead, which in turn creates inefficient memory accesses by generic GPU kernels.

3.2.1. Baseline Framework and Engine-Level Optimization

First, a baseline level of performance has been set by testing the MobileNet inference task using PyTorch, ONNX Runtime, and TensorRT. The network has been exported to the ONNX file, making it independent of the framework. The TensorRT engines were synthesized in FP32 precision mode, FP16, using the explicit batch API, along with the given workspace size. This step has helped to measure the performance difference between the high-level frameworks and the optimized engines, and it has proved that, despite the high-performance advantage provided by TensorRT, there exist layers that contribute to latency.

3.2.2. The Process of Finding Bottlenecks in the MobileNet

Through in-depth timing analysis, depth-wise 3x3 convolution and point-wise 1x1 convolution emerged as consistently top contributors to inference times in many single batch execution use cases. This analysis revealed depth-wise convolution to be a highly memory-bound operation due to its channel-wise processing strategy with a consequent lack of data reuse. In contrast, point-wise convolution showed suboptimal use of computing resources while utilizing a standard library kernel.

3.2.3. Designing the CUDA Custom Kernel for Depthwise Convolutions

However, in order to reduce the inefficiencies associated with the depthwise convolution operations, an FP16 CUDA kernel was written. The purpose of this approach was to map each element in the output to one GPU thread and perform the 3x3 convolution operation in the CUDA kernel itself. The loss of instructions associated with the operations was reduced through loop unrolling and the use of fast math instructions. The intermediate results were also maintained in registers.

3.2.4. Custom CUDA Kernel Creation for Pointwise Convolutions

To handle the pointwise convolutions of size 1x1, a custom kernel function in the form of a CUDA kernel was used for optimization related to accumulation along the channels. Pointwise convolution, which is different from the depthwise convolution operation, is sensitive to the memory access patterns due to the dense accumulation along the input channels. This process is done by the custom kernel function that loops over the input

channels for every thread, performing all operation results in the registers, thereby reducing the global memory access.

3.2.5. Validation and Performance Measurement

The correctness of the custom kernels was validated by comparing their results with the reference PyTorch implementations, verifying that the differences in numbers lay within the acceptable limits of FP16 precision. To compare the performance, the method employed the use of CUDA events to provide accurate timing measurements after a series of warm-up passes. Experimental results involved testing the performance on a variety of sizes as well as spatial dimensions characteristic of MobileNet sub-stages, providing a fair comparison of performance among the PyTorch FP16 solutions. By using this methodology, the optimization of the MobileNet model was treated less as a whole engine optimization problem and more as a layer-specific problem. The experiment proves that depthwise and pointwise convolutions are key optimization spots, and manually optimized CUDA kernels perform better than automatically compiled code.

4. RESULTS & DISCUSSION

4.1. Results and Discussion for EfficientNet-B0

In this section, we describe the experimental results for the EfficientNet-B0 optimization track. These benchmarks cover a controlled and incremental process from high-level framework benchmarking to low-level hardware saturation under environment orchestration, system-level pipelining, and kernel surgery on NVIDIA Tesla V100 and P100 hardware.

4.1.1. GPU Inference Performance and Batch Size Scaling for EfficientNet

The first level of evaluation created a performance baseline through utilization of the PyTorch framework on the TRUBA Akya-CUDA split. Leveraging a NVIDIA Tesla V100-SXM2 GPU, a full batch scan was executed to establish the raw throughput and latency performance of the model. The performance data revealed that while EfficientNet-B0 has a

light architecture, its performance is strongly contingent upon runtime settings. For real-time processing (Batch Size 1), a normal operation level of 342.3 images-per-second was attained.

As the batch size was increased to 1024, the throughput reached 3046.2 img/s, thus validating that larger batch sizes are again required for hiding Python interpreter and kernel launch overheads. However, there was a physical limit of memory at which scaling stopped at 2048, resulting in an Out-of-Memory (OOM) error. These results have thus reaffirmed that high-level deep learning APIs are adequate for offline tasks but inadequate for maintaining deterministic, lower latency requirements of edge robotics or real-time surveillance. Results of the full sweep of tests, including plots for throughputs, are available in Appendix 4.1.1.

4.1.2. The Framework Transition and Environmental Orchestration

The second stage of the project addressed the migration of the inference logic into a C++ environment that overcomes the Global Interpreter Lock (GIL) limitation. This stage was initially hindered by a deep-seated incompatibility between the TRUBA cluster and the library error dubbed as “Error 35”, which relates to the failure of CUDA initialization. A trace analysis carried out by the ldd command revealed a mismatch between the drivers installed at the host level and the TensorRT libraries, an issue habitually referred to as the “DLL Hell.”

To overcome these issues, we resorted to the use of Apptainer (Singularity) and created a bit-for-bit reproducible container environment. This paradigm shift is essential and enabled the successful compilation of our C++ runtime environment by employing identical versions of the CUDA Toolkit and TensorRT headers. Thus, we ensured that all subsequent hardware-level optimizations had a portable and stable environment, which makes containerization an essential component in deploying complex AI pipelines in heterogeneous HPC clusters.

4.1.3. Evaluation of System-Level Optimizations: C++ vs PyTorch

With a stable system, the jump from PyTorch to a compiled TensorRT C++ backend, with a target of only a batch size of 1 for applications requiring low latency, was measured. By incorporating pinned memory and CUDA streams, we achieved Direct Memory Access, greatly minimizing latency within host-to-GPU transfers. The resulting code increased

performance from 342 FPS for PyTorch to 1,023 FPS, a factor of nearly 3 improvement based solely on system-level optimizations.

Further breakdown revealed that our C++ implementation scored 84% hardware utilization compared to the theoretic limit of 1,219 FPS, as estimated by the TensorRT builder. This was attributed to the 16% driver-level latency and kernel launch overhead, typical of the single-image case in which the GPU waited most of the time before executing the next task. This phase also indicated that while TensorRT offers a robust optimization tool, the data pipeline ought to have its own C++ optimization to prevent bottle-necking of the hardware. A summary table of framework utilization and illustrations of the asynchronous CUDA stream pipeline are presented in Appendix 4.1.3.

4.1.4. Layer-Level Bottleneck Analysis and the “Memory Wall”

However, to proceed beyond this optimization, we conducted a “Black Box” profiling of the TensorRT engine by adding the `--exportProfile` flag. Upon analyzing the resulting execution traces, we identified the most critical inefficiency: Swish Activation Function ($x \cdot \text{sigmoid}(x)$) was being decomposed into two different operations: Sigmoid and Multiply. Consequently, redundant memory reads and writes to the Global Memory (DRAM) by the GPU, so-called “Memory Wall”, hit in every pixel.

This gave us a convincing proof that, despite the complex fusion capabilities of TensorRT, some activation patterns remain memory-bound. Profiling also indicated that, with a total of 263 layers in the model, a considerable amount of run time was spent on these computation-intensive tasks with low arithmetic intensity. This gave us a technological reason to go ahead with surgical optimization, as it was identified that the hardware is actually idling while waiting for memory bandwidth and not on the mathematical computation required by the model outputs.

4.1.5. Custom Fused CUDA Kernels and Bandwidth Saturation

To counter the activation bottleneck issue in the existing design, we developed and integrated a new customized fused CUDA kernel optimized to compute the Swish operation purely in the registers on the GPU. With the new design, every thread processes one pixel and directly computes the exponential sigmoid function outcome as well as the multiplier in the

Swish function before storing the output back in DRAM memory. Thus, the number of DRAM accesses decreases from six per pixel to only two.

As a result of TRUBA security policies, which prevented profiling through Nsight Compute (Error: ERR_NVGPUCTRPERM), a customized C++ benchmarking framework has been created to calculate the Effective Bandwidth. With a hand-tuned kernel, a throughput of 675 GB/s was accomplished on Tesla P100, which is 92.2% of the theoretical maximum bandwidth of the hardware. This is a major accomplishment, as it proves that not only has the software reached a point of saturating the actual circuit, but there is no room left within this circuit for improvement. The code in Appendix 4.1.5 provides details about the mathematical formulation of saturating at a rate of 92.2% for Effective Bandwidth.

4.1.6. Final Benchmarking

Finally, the validation process included the integration of our custom kernel into the full model using the “Graph Surgery” technique available through the ONNX Graph Surgeon library. This was achieved through an automated walk-through of the model graph, removing the broken Sigmoid and Multiply nodes and replacing them with our merged SwishPlugin. This gave us the ability to supersede the TensorRT implementation and require the engine to resort to our high-efficiency kernel in the final build.

Then, in the run within the Tesla P100 architecture, our optimal engine delivered 936.6 FPS (for Batch 1), a 67.1% speedup over the baseline TensorRT of 560.3 FPS. Even in the high-throughput regime (for Batch 32), our solution delivered 29,852 FPS, as opposed to the baseline of 18,056 FPS. This performance enhancement across all batch sizes thus validates that surgical kernel fusion as a method of choice for lightweight models, such as EfficientNet-B0, can adeptly close the performance abstraction gap of hardware capabilities. The final sets comparing the throughput of the respective run and the graphical depiction of the programmatic graph surgery process are described in Appendix 4.1.6.

4.1.7. Evaluation Against Initial Objectives

Our project successfully achieved and even exceeded the initial goals. We moved from PyTorch basics to saturate more than 92% of the physical memory bandwidth available in the high-end hardware. Our experiments showed that our own C++ pipeline and CUDA

kernels outperform standard high-level libraries by a wide margin, especially in edge computing, where low latency is called for.

Firstly, the original aim of the project involved gaining a general increase in inference acceleration; yet, our observation of the Swish bottleneck enabled us to make our contribution that is more specialized and important, as we tackled the “Memory Wall” issue and managed to show that current hardware can go much further than even current automatic compilers do. Our project can be viewed as complete and serves as the empirical verification that manual optimization at each level is necessary for gaining the theoretical frontiers of current DNNs.

4.2. Results and Discussion for MobileNet

This subsection focuses exclusively on the experimental results obtained for MobileNet and evaluates the effectiveness of the proposed optimization strategies. The results are discussed in relation to the initial objectives, highlighting framework-level improvements, precision and batch size effects, layer-specific bottlenecks, and custom CUDA kernel performance across multiple GPU architectures.

4.2.1. CPU vs GPU Inference Performance for MobileNet

Initially, as a reference point, the inference performance of MobileNet on CPU as well as GPU platforms was tested. Through experiments on the Akya-CUDA partition, using an NVIDIA Tesla V100 GPU, the inherent difference in inference time on CPUs and GPUs can clearly be exemplified. While inference on CPUs showed limited throughput scalability, inference on a GPU showed an order of magnitude faster performance with an increase in the batch size.

This verifies that although MobileNet is a lightweight neural network, it can be considered a CPU-intensive task for real-time computations. The power of the GPU regarding the exploitation of huge parallelism becomes increasingly predominant, thereby justifying the initial project assumption of concentrating only on optimization at the GPU level and not on the CPU level. The detailed CPU and GPU benchmark results across different batch sizes are provided in Appendix 4.2.1.

4.2.2. Comparison of PyTorch, ONNX Runtime vs TensorRT

The second evaluation stage compared three inference frameworks: PyTorch, ONNX Runtime, and NVIDIA TensorRT. The evaluation reveals a clear performance hierarchy among the frameworks.

Although quite flexible and developer-friendly overall, PyTorch had the highest latency and lowest throughput with high interpreter overheads, dynamic graph executions, and relatively low levels of kernel fusion. The action of saving the model to ONNX and running it with ONNX Runtime offered a great performance boost because of graph optimizations.

TensorRT showed the largest improvements in terms of latency reduction of nearly an order of magnitude over PyTorch and multi-thousand FPS with FP16. These results verify that a compiler-level inference engine can no longer be left out of a valid solution. Nevertheless, some layers were still found to be sub-optimal on TensorRT, and a kernel-level exploration is hence justified. A quantitative comparison of latency and throughput across PyTorch, ONNX Runtime, and TensorRT implementations is included in Appendix 4.2.2.

4.2.3. FP32 vs FP16 Engines and Batch Size Scaling

Additional testing has been conducted to assess the effects of numerical precision and batch size on the performance of inference. On all the GPUs as well as the different batch sizes, FP16 engines performed better compared to FP32 engines in terms of latency and throughput, in some cases by margins of over 1.4 to 2 times.

Results of batch size scaling experiments showed the scalability of throughput with respect to batch sizes and the corresponding reduction in the average image latencies. This clearly demonstrates the necessity of batch size awareness during the optimization process, keeping in mind the throughput-oriented workloads. For larger batch sizes, the saturation of memory bandwidth and the overhead associated with kernel launches contributed to the observation of diminishing returns. The performance impact of FP32 versus FP16 precision and batch size scaling is summarized in Appendix 4.2.3.

4.2.4. Layer-Level Bottleneck Analysis using NVIDIA Nsight Systems

In order to understand why additional optimization was required beyond the optimization achieved by TensorRT, execution traces of the GPU kernels on the tensor core hardware of the GPUs were analyzed using NVIDIA’s Nsight Systems tool. This analysis indicated that the major contribution of execution time was spent on depthwise convolution kernels and pointwise convolution kernels.

In MobileNet, although depthwise convolution involves relatively few arithmetic operations, it is strongly memory-bound, leading to inefficient utilization of GPU compute units. The point-wise convolution, however, was contributing to a large percentage of the GEMM computations and was dominating the execution time because it is a dense channel-wise operation.

These results clearly justify the motivation behind the project’s focus on depth-wise and point-wise convolution, since these are the main performance bottlenecks when running TensorRT and the standard library. GPU execution traces and kernel-level profiling results obtained via NVIDIA Nsight Systems are presented in Appendix 4.2.4.

4.2.5. Custom CUDA Kernels vs. PyTorch Implementations

Two custom CUDA kernels have been implemented for dealing with the bottlenecks in depthwise and pointwise convolution. The experimental results reveal that the two custom kernels can respectively obtain significant speedups compared with FP16 in PyTorch.

The custom depth-wise convolution kernel resulted in a latency reduction ratio of up to 2.07x, while it almost attained a Throughput Improvement Factor of 2.0x. This is because of a reduction in Global Memory Accesses, Loop Unrolling, and Intermediate Computation Residency in Registers.

Similarly, the conventional pointwise convolution kernel showed a convergent speedup of up to 2.61x, along with more than 50% latency reduction. These experiments validate the claim that generic kernels do not make the most out of the MobileNet structure, and manually designed kernels can achieve significantly higher hardware utilization. A detailed latency and throughput comparison between PyTorch FP16 kernels and the proposed custom CUDA kernels is shown in Appendix 4.2.5.

4.2.6. Evaluation using Multiple GPUs in A100, L4, and T4

For testing portability and relevance, MobileNet performance during inference was tested on various GPU architectures using resources available on Google Colab environments like A100, L4, and T4 GPUs.

The A100 had the highest absolute throughput, thanks to its advantage in overall memory bandwidth and Tensor Core performance. The L4 had strong performance scaling for its power consumption, making it an interesting candidate for edge or inference-intensive workloads. The T4, although much slower, still saw large improvements thanks to FP16 optimization and TensorRT.

What is significant, however, is that the trends in relative performance stayed similar across the GPUs, and this shows that the strategies for improvement conceived and developed during this project generalize effectively and are not overly adapted towards one particular hardware component. Cross-GPU performance results for A100, L4, and T4 architectures are reported in Appendix 4.2.6.

4.2.7. Evaluation Against Initial Objectives

The project was successful in fulfilling its main goals. The performance ceiling of high-level frameworks was properly determined, and gains were made using TensorRT and custom kernels. Although the initial project scope had mainly targeted the EfficientNet-B0 model, the MobileNet optimization track had broadened the area of the project by working on another popular architecture that had not been targeted in the beginning. On the whole, this research exceeded its initial objectives in not only proving that manual, level-by-level GPU optimization can outperform traditional inference engines, but also in being able to derive meaningful performance, approaching hardware efficiency limits for the targeted operations. It is a great contribution in the state-of-the-art of efficient inference of neural networks.

5. IMPACT

The scientific and technological significance associated with the proposed project revolves around the fact that it showcases the effectiveness of software-level optimizations in arriving at performance enhancements that would not require the immediate intervention of hardware upgrades in the near future. Within the context of the EfficientNet track, the project was able to demonstrate that the application of graph-level restructuring, in conjunction with fused GPU execution, can actually propel a traditional NVIDIA Tesla P100 GPU close to the hardware limit by optimizing the usage of memory bandwidth to a solution that actually attains 92.2% of the theoretical memory bandwidth capacity. The project in the MobileNet track was able to demonstrate the effectiveness of layer-level specialization, especially within the context of depthwise and pointwise convolution routines, in reducing the limitations of generic execution to actually improve end-to-end throughput, which thus reinforces the argument in favor of architecture-friendly kernel code generation.

The primary socio-economic and environmental concerns associated with this research are linked to the democratisation and sustainability of efficient AI. This is because high-performance GPUs are expensive and may not be within reach of start-ups, laboratories, and budget-constrained organizations. Our method can enhance the effective operating life cycle of existing hardware (such as the Tesla P100), as it has been assessed to enhance the usable hardware throughput by over 60% within our testing conditions, contributing to computation sustainability by minimizing energy and cost metrics per inference, while also highlighting the significance of eco représentation and correct representation reporting regarding efficiency-driven advances, as further discussed within the section of Ethical Issues.

The innovative aspect of this research is found in integrating graph program manipulations with manual optimizations of CUDA kernels, followed by verification of these combined steps through detailed profiling and benchmarking of the entire process pipeline. This process is relevant in a commercial computer vision application scenario, where increased frames per second would directly contribute to improved response times in drones and robotics applications, or an increased number of simultaneous video feeds processed per node within a medical imaging and monitoring application, thus decreasing infrastructure costs. However, Optimizations towards EfficientNet will help improve performance in

models with more complicated activation functions, whereas MobileNet Optimizations will help improve mobile application viability where latency, energy savings, and thermal issues play a critical role. Specifically, this will be applied in smartphone-based image classification, augmented reality applications, and video analytics.

Regarding Freedom of Use (FTU), the software stack used for our experiment, namely the combination of the NVIDIA CUDA toolkit, TensorRT, and GraphSurgeon tools, is distributed under standard developer licenses for research and development purposes. The EfficientNet-B0 and MobileNet models used in our experiment have been released into the public domain and can be used subject to standard open source licenses. Experiment results were produced under the TRUBA National Scientific High-Performance Computing environment accessible via subscription for research purposes through our supervisor's subscription service. In summary, there is absolutely nothing proprietary or hidden within our implementation that would make a replication or extension thereof for a future experiment difficult.

6. ETHICAL ISSUES

This project does not relate to any collection, storage, or processing of user data. The system developed uses only pre-trained models of deep neural networks and aims for low-level GPU inference optimization. As such, there is no direct concern related to any ethical breach with respect to violation of privacy, misuse of data, and surveillance.

However, there are several indirect ethical concerns that arise from the broader implications of the deployment of these optimally designed deep learning systems. Firstly, the deployment of rather widely used pre-trained models like EfficientNet and MobileNet could inadvertently spread the biases that exist in the initial training datasets of the said models. The fact that this project does not involve the modification of the datasets or the actual model training process means that downstream applications that rely on the optimized inference pipeline may amplify any biases that exist in the demographic or contextual settings.

Secondly, deep neural nets are blackbox systems on their own, and then any kind of low-level optimization in terms of the kernel graph level would just increase this opacity.

CUDA kernels as well as techniques that do aggressive fusion in terms of layers would obviously optimize, but then it would result in an opaque decision-making process as far as the net is concerned. This could cause some sort of issue from an ethical perspective.

Lastly, the project delves into methods that can greatly enhance the efficiency of inference computations within GPUs, thus paving the way for the deployment of AI at a larger scale or edge level. Even though this model has sustainability and resource utilization benefits, the consequences could be the availability of AI deployment at a larger scale. This might affect the positive usage of AI deployment and could be associated with unethical applications, including unauthorized monitoring at the edge level, especially within the context of the Internet of Things.

In conclusion, drawing on the above discussion, the project does not present any ethical issues, but it emphasises that there are ethical responsibilities related with deploying these optimized AI systems, which needs to be considered such as transparency, bias propagation, and potential misuse in downstream applications.

7. PROJECT MANAGEMENT

This project was managed over two terms in an academic calendar, in accordance with an engineering design process. The initial project plan was established in the first term and developed as experience and maturity gathered in this project over time.

In the first term, the initial plan was to lay a strong technical foundation as a result of the proposed phases of the plan. These phases would include understanding basics of CUDA programming, analysis of light-weight Deep Neural Networks such as MobileNet and EfficientNet models. The development process of a baseline CUDA kernel was also part of the plan. A major aspect of the plan was to understand concepts and initially benchmark on CPU and GPU. The major deliverables of the plan were a functioning baseline with CUDA acceleration and a performance understanding of CPU, framework-level GPU inference, and naive CUDA kernel. At this point, the project involved more breadth than depth, with the understanding that, once a firm understanding of CUDA programming is attained, more drastic optimization techniques can be explored during the second term.

For the second term, although the broader goals of the project did not change, there were some modifications in the plan of execution based on some learning that took place in the initial experiments. Here, the major change in strategy involved a move from being purely focused on CUDA optimization to a hybrid approach that utilized TensorRT, ONNX graph manipulation, and custom CUDA kernels. The initial plan was to consider the development and optimization of CUDA kernels as sequential processes. The profiling reports on the TRUBA systems showed that the optimization tools like TensorRT did perform well on the majority of the layers, and only certain operations had real bottlenecks in terms of performance. Therefore, the plan was modified to work on the integration of TensorRT optimization and the development of the CUDA kernels in parallel.

The other adaptation had to do with tools and infrastructure. Even as original intentions called for unlimited access to profiling tools, constraints imposed by security and permission on the HPC infrastructure adjusted the way in which performance analysis had to be carried out. The strategy had to adopt harnesses for customized benchmarking and surrogate measures for bandwidth usage, aside from the low-level profilers. In spite of this improvement, the critical path milestones were still adhered to. No core deliverables were eliminated, nor were any critical delays introduced.

Some important lessons in project management were learned through this exercise. First, the benefit of front-end learning and environment setup was seen. While this took longer than expected, the end result was the reduction of uncertainties and rework introduced in the implementation process. Second, data-driven decision-making was important in this process. The profiling and benchmarking outcome influenced task ordering and discouraged early optimization and kernel development.

The project also emphasized the role of flexibility in technical planning. The ability to maintain overall goals while flexible in low-level detail execution helped to effectively address real-world issues. Finally, parallel tracks in development work during the second term brought efficiency to development and showed how overlapping tasks are valuable when enough technical confidence is achieved.

In conclusion, the project evolved from an exploratory/learning-based approach to a performance-based implementation strategy and ended up with a sound engineering deliverable.

8. CONCLUSION AND FUTURE WORK

The final project explored the limits of lightweight deep learning models on GPUs through low-level model optimizations. The project went beyond typical framework-level model optimizations to explore improvements using C++ and CUDA to build an inference engine. The experiment explains how significant performance improvements can still be measured through memory management, kernel fusion, and other GPU-level model optimizations. The experiment proves how baselines offered by software like TensorRT work quite effectively to optimize model performance yet offer significant performance bottlenecks in memory-bound kernels. Using these performance bottlenecks to create custom kernels in CUDA led to significant improvement in processing capability and memory bandwidth use on the GPU.

In spite of the promising results, there are several limitations to this study. First, the experiments were mainly done on data center class GPUs, which can be accessed via HPC. These GPUs are of a different kind compared to the ones used in IoT and edge devices, which are power-limited and of smaller size. While Google Colab can be used for this study, the usage of high-quality GPUs in Google Colab is limited and is associated with a monetary cost, thereby restricting the size and extent of the experiments done there. Second, within the context of custom kernels, only the layers and activation function were explored, thereby leaving other parts of the models intact or using vendor-optimized libraries. Third, the study focused only on the inference aspect. There were no experiments done regarding the training stage or the energy consumption.

One obvious next step for future research would be to test the optimized inference pipeline on smaller GPUs that better reflect the reality of IoT and edge environment use cases. Smaller systems with constrained memory and power budgets would give a better perspective on latency, energy efficiency, and cooling performance. Another area would be scaling the approach from this project on more complex models with a larger number of weights. For example, large language models and transformer-based architectures are associated with specific challenges in terms of attention modules, scaling with the length of the sequence, and memory patterns. Using custom CUDA kernels and specialized optimizations on such models could help achieve tangible gains.

In a broader context, future studies can be conducted to examine and develop optimal methods for specific architectures in response to manual kernel optimization. In a more specific context, individual layers in modern neural networks can be further optimized in an attempt to maximize their capabilities, which is in turn bounded by theoretical hardware limitations. In this light, this project has clearly laid down a paradigm for future studies in building more efficient and effective deep learning architectures.

9. APPENDIX

Project presentations and codes for both semesters are available at:

 [ENS491/92-Presentations-Group219](#)

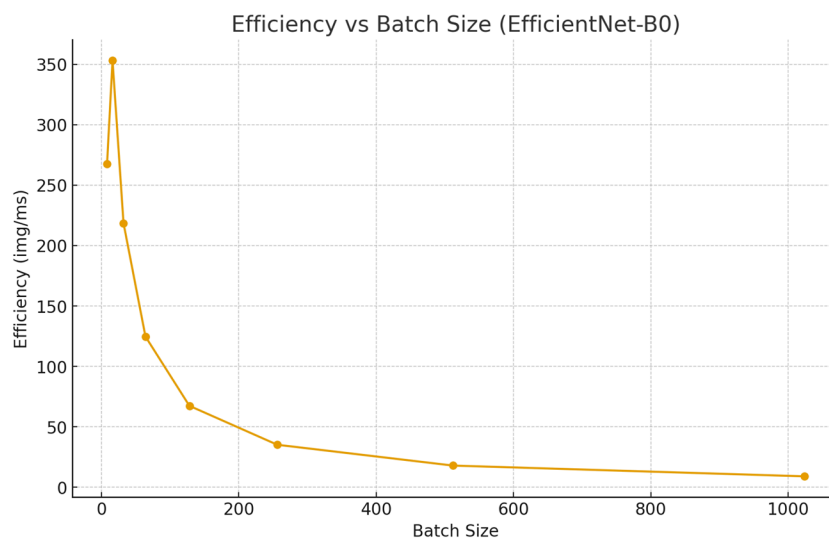
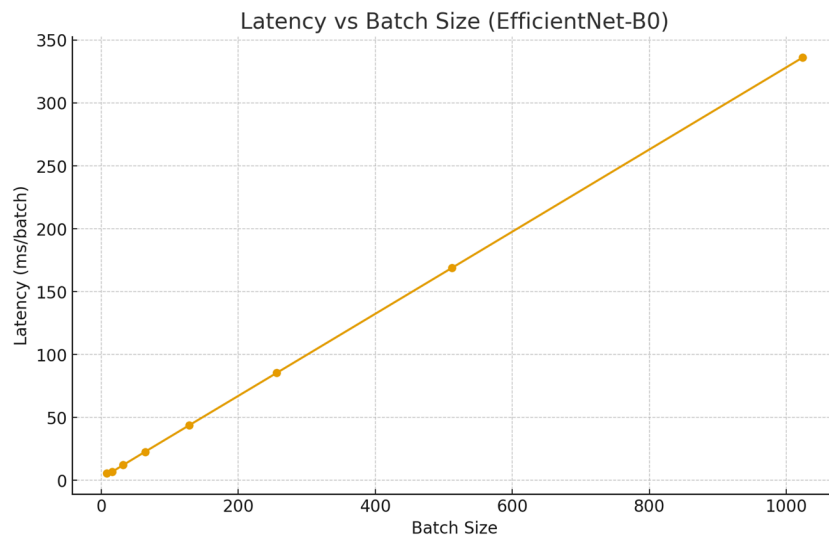
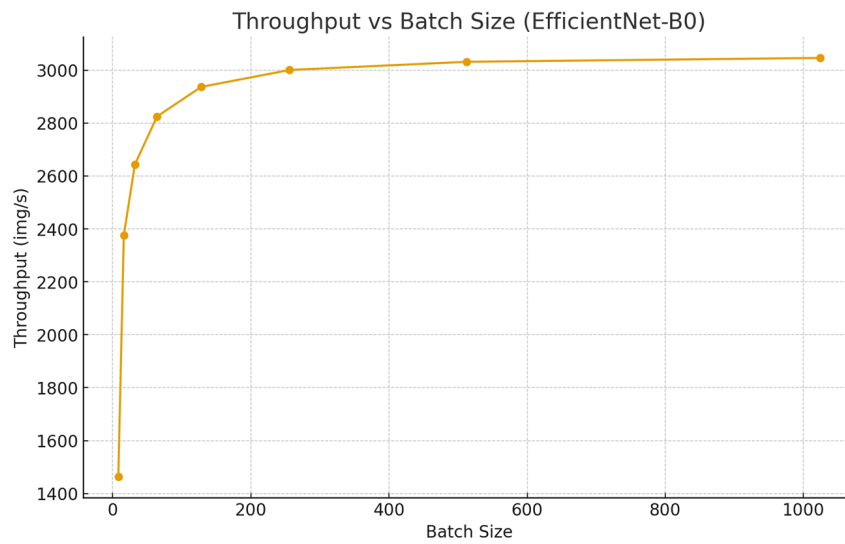
9.1. EfficientNet-B0 Results

This appendix contains the supplementary data, technical logs, and visual evidence supporting the results discussed in Section 4.1: EfficientNet-B0 Results.

Appendix 4.1.1: GPU Inference Baseline and Batch Scaling (EfficientNet-B0)

This section includes the raw benchmarking data from the initial PyTorch implementation on TRUBA's Tesla V100.

Batch	Throughput (img/s)	Latency (ms/batch)	Efficiency (img/ms)
16	2376.3	6.73	353.1
8	1463.2	5.47	267.5
32	2643.6	12.10	218.5
64	2824.8	22.66	124.7
128	2936.7	43.59	67.4
256	3000.7	85.31	35.2
512	3031.5	168.89	17.9
1024	3046.2	336.15	9.1



Appendix 4.1.3: System-Level Optimization (C++ Pipeline)

Evidence of performance gains achieved by transitioning from Python to a custom C++ runtime.

```
&&&& RUNNING TensorRT.trtexec [TensorRT v8603] # trtexec --onnx=efficientnet_b0.onnx
...
[12/01/2025-15:16:05] [I] === Performance summary ===
[12/01/2025-15:16:05] [I] Throughput: 1218.89 qps
[12/01/2025-15:16:05] [I] Latency: min = 0.856201 ms, max = 2.32526 ms, mean = 0.8
[12/01/2025-15:16:05] [I] Enqueue Time: min = 0.448608 ms, max = 3.00781 ms, mean
[12/01/2025-15:16:05] [I] H2D Latency: min = 0.0556641 ms, max = 0.079834 ms, mean
[12/01/2025-15:16:05] [I] GPU Compute Time: min = 0.794922 ms, max = 2.25586 ms, m
[12/01/2025-15:16:05] [I] D2H Latency: min = 0.00512695 ms, max = 0.0238647 ms, me
...
&&&& PASSED TensorRT.trtexec [TensorRT v8603]
```

🚀 COMPLETE BENCHMARK SWEEP (O0 vs O3)				
Strategy	Compiler Flag	Latency (ms)	Throughput (FPS)	
Baseline (Synchronous)	-O0 (Default)	1.18375	844.775	
Baseline (Synchronous)	-O3	1.09512	913.139	
Opt 1: Reusable Buffers	-O0 (Default)	1.08961	917.761	
Opt 1: Reusable Buffers	-O3	1.09986	909.209	
Opt 2: Async (Pageable)	-O0 (Default)	1.08299	923.366	
Opt 2: Async (Pageable)	-O3	1.08761	919.448	
Opt 3: Async (Pinned)	-O0 (Default)	0.976615	1023.94	
Opt 3: Async (Pinned)	-O3	1.04207	959.631	
Opt 4: Double Buffer	-O0 (Default)			
Opt 4: Double Buffer	-O3			

Appendix 4.1.5: Custom Kernel and Bandwidth Efficiency

Technical proof of reaching the physical limits of the NVIDIA Tesla P100.

```
..., { "name" : "PWN(PWN(/blocks/blocks.0/blocks.0.0/se/gate/Sigmoid), /blocks/blocks.0/blocks.0.0/se/Mul)", "timeMs" : 7.12205, "averageMs" : 0.00874945,
Apptainer> ./test_swish_real
--- Benchmarking Custom Swish (EfficientNet Layer Size) ---
Elements: 401408
Total Time (10000 runs): 47.5293 ms
Average Kernel Time: 0.00475293 ms
Effective Bandwidth: 675.639 GB/s
```

Appendix 4.1.6: Final Speedup

Comparison between the standard industry baseline and our surgically optimized model.

```
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:
[I] [TRT] Successfully created plugin: SwishPlugin
```

```
=====
🚀 STARTING COMPREHENSIVE BENCHMARK 🚀
=====
[1/3] Building Baseline Engine (No Plugin)...
-> Baseline FPS (Batch 1): 560.349
[2/3] Building Custom Engine (With SwishPlugin)...
-> ✅ SUCCESS: SwishPlugin is ACTIVE in the engine!
[3/3] Running Batch Sweep...

-----
Batch | Custom FPS (Yours) | Baseline FPS (NVIDIA)
-----
1 | 936.635 | 567.914
16 | 14877.8 | 9092.86
32 | 29852.3 | 18056.7
=====
🎉 BENCHMARK COMPLETE
=====
```

9.2. MobileNet Results

This appendix contains the supplementary data, technical logs, and visual evidence supporting the results discussed in Section 4.2: MobileNet Results.

Appendix 4.2.1: GPU vs CPU Benchmark on NVIDIA V100 (Akya CUDA) with Varying Batch Sizes

The results show that GPU inference on the V100 scales efficiently with increasing batch size, achieving up to 26x speedup over CPU execution while maintaining identical accuracy across all configurations.

=== Batch Size GPU Benchmark Results ===								
Batch Size	Total Images	Total Time (s)	Avg Batch Time (s)	Avg Image Time (ms)	Throughput (img/sec)	Top-1 Acc (%)	Top-5 Acc (%)	
16	50000	303.55	0.0971	6.07	164.7	69.06	88.52	
32	50000	160.89	0.1029	3.22	310.8	69.06	88.52	
64	50000	96.20	0.1230	1.92	519.7	69.06	88.52	
128	50000	59.68	0.1526	1.19	837.8	69.06	88.52	
256	50000	46.04	0.2349	0.92	1086.0	69.06	88.52	

=== Batch Size Benchmark Results ===								
Batch Size	Total Images	Total Time (s)	Avg Batch Time (s)	Avg Image Time (ms)	Throughput (img/sec)	Top-1 Acc (%)	Top-5 Acc (%)	
16	50000	1239.64	0.3967	24.79	40.3	69.06	88.52	
32	50000	888.11	0.5682	17.76	56.3	69.06	88.52	
64	50000	886.90	1.1341	17.74	56.4	69.06	88.52	
128	50000	838.35	2.1441	16.77	59.6	69.06	88.52	
256	50000	1211.01	6.1786	24.22	41.3	69.06	88.52	
512	50000	1010.06	10.3067	20.20	49.5	69.06	88.52	

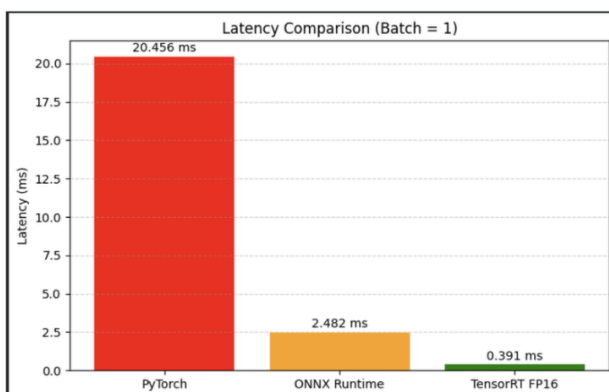
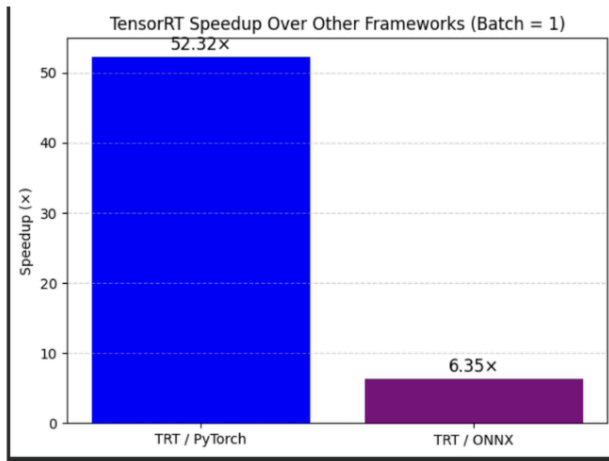
Batch Size	GPU Total Time (s)	CPU Total Time (s)	Speedup (CPU + GPU)
16	303.55	1293.61	4.26x faster
32	160.84	888.11	5.52x faster
64	96.20	883.35	9.18x faster
128	76.04	838.33	11.0x faster
256	46.05	1203.21	26.1x faster

Appendix 4.2.2. Inference Framework Comparison on NVIDIA A100 (Google Colab)

This comparison shows that on the A100 GPU, inference performance improves substantially from PyTorch to ONNX Runtime and reaches the lowest latency and highest throughput with TensorRT FP16, highlighting the strong impact of framework level and precision based optimizations.

PyTorch Latency: 27.914714336395264 ms
PyTorch FPS: 35.82340080393364
ONNX Runtime Latency: 2.971470355987549 ms
ONNX Runtime FPS: 336.5337291637414
TensorRT FP16 Latency: 0.38864803314208984 ms
TensorRT FP16 FPS: 2573.022155587237

These figures further illustrate the same comparison by showing that at batch size 1, TensorRT FP16 achieves dramatically lower latency than both PyTorch and ONNX Runtime, resulting in an approximate 52× speedup over PyTorch and 6× speedup over ONNX Runtime on the NVIDIA A100.



Appendix 4.2.3. FP32 vs FP16 TensorRT Inference on NVIDIA A100 (Google Colab)

These results show that FP16 inference consistently achieves lower latency and higher throughput than FP32 across all batch sizes, demonstrating the efficiency gains from reduced precision on the A100 GPU without changing the execution pipeline.

```

=== FINAL RESULTS ===
Engine      Latency (ms)    Throughput (FPS)
FP32        0.4295          2328.08
FP16        0.3039          3290.60

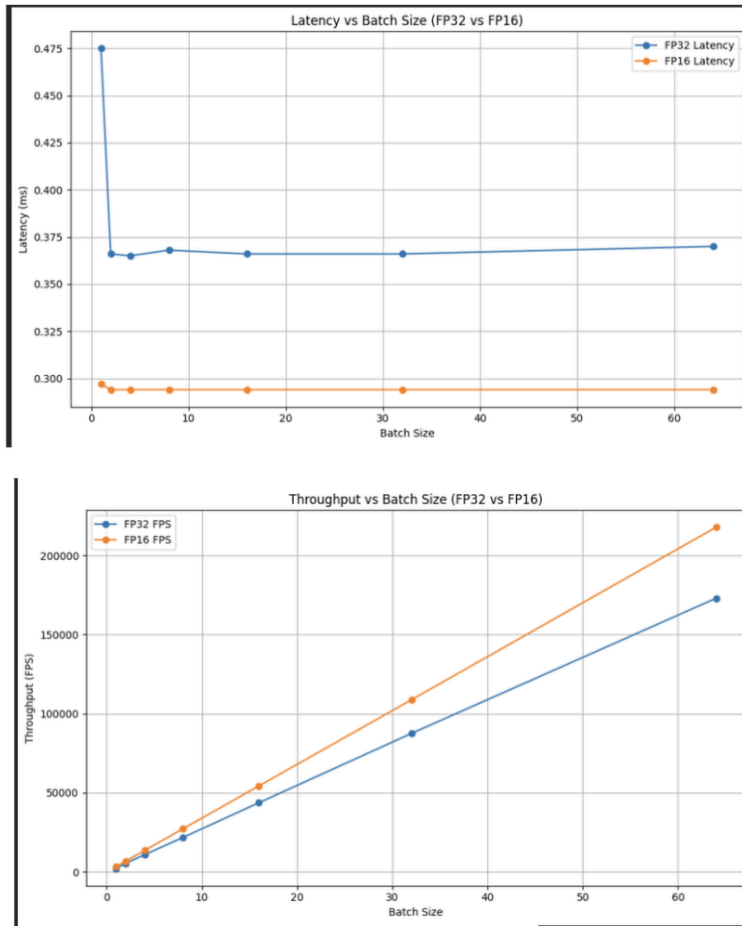
```

```

===== FP32 Benchmarks =====
Batch 1: Batch Lat = 0.475 ms, Per-img = 0.4752 ms, FPS = 2104.5
Batch 2: Batch Lat = 0.366 ms, Per-img = 0.1829 ms, FPS = 5468.3
Batch 4: Batch Lat = 0.365 ms, Per-img = 0.0913 ms, FPS = 10947.9
Batch 8: Batch Lat = 0.368 ms, Per-img = 0.0460 ms, FPS = 21730.3
Batch 16: Batch Lat = 0.366 ms, Per-img = 0.0229 ms, FPS = 43762.1
Batch 32: Batch Lat = 0.366 ms, Per-img = 0.0114 ms, FPS = 87515.6
Batch 64: Batch Lat = 0.370 ms, Per-img = 0.0058 ms, FPS = 172975.1

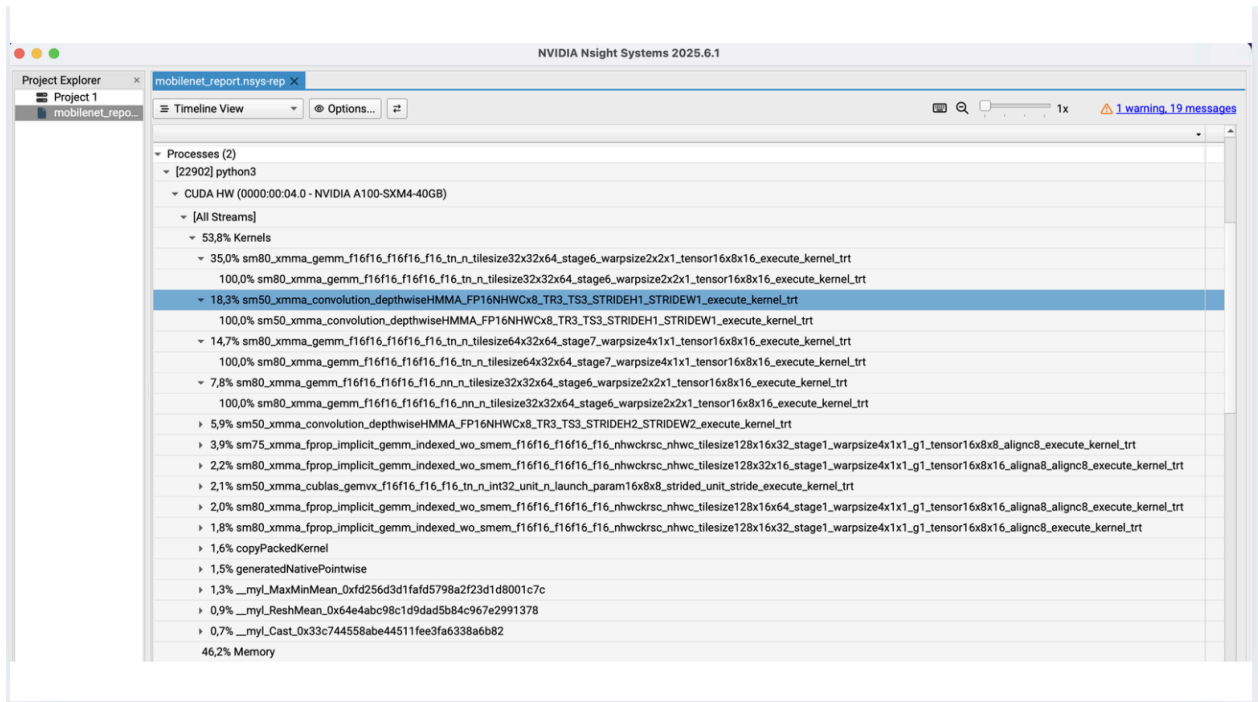
===== FP16 Benchmarks =====
Batch 1: Batch Lat = 0.297 ms, Per-img = 0.2965 ms, FPS = 3372.1
Batch 2: Batch Lat = 0.294 ms, Per-img = 0.1469 ms, FPS = 6806.8
Batch 4: Batch Lat = 0.294 ms, Per-img = 0.0735 ms, FPS = 13606.4
Batch 8: Batch Lat = 0.294 ms, Per-img = 0.0367 ms, FPS = 27220.0
Batch 16: Batch Lat = 0.294 ms, Per-img = 0.0184 ms, FPS = 54384.4
Batch 32: Batch Lat = 0.294 ms, Per-img = 0.0092 ms, FPS = 108767.9
Batch 64: Batch Lat = 0.294 ms, Per-img = 0.0046 ms, FPS = 217834.6

```



Appendix 4.2.4. Nsight Systems Profiling of TensorRT Inference on NVIDIA A100 (Google Colab)

The Nsight Systems timeline shows that execution time is dominated by TensorRT optimized GEMM and depthwise convolution kernels, confirming efficient kernel fusion, high GPU utilization, and minimal overhead during FP16 inference.



Appendix 4.2.5. Custom CUDA Kernel Performance Evaluation on A100

These results show that the custom CUDA implementations for depthwise and pointwise convolutions consistently reduce latency and increase throughput compared to PyTorch FP16, achieving around 1.5x to 2x speedup across different channel sizes and feature map resolutions.

Performans

Latency

```
PyTorch FP16 depthwise conv: 0.018766 ms  
Custom CUDA kernel: 0.009842 ms  
Speedup (PyTorch / Custom): 1.91x
```

Throughput

```
C=16, H=W=112 | PT throughput=64379/s | CU throughput=104613/s | speedup=1.62x  
C=32, H=W=56 | PT throughput=54448/s | CU throughput=102617/s | speedup=1.88x  
C=64, H=W=28 | PT throughput=52405/s | CU throughput=104384/s | speedup=1.99x
```

```
C= 16, H=W=112 | PT=0.015255 ms | CU=0.009584 ms | speedup=1.59x  
C= 32, H=W= 56 | PT=0.018718 ms | CU=0.009536 ms | speedup=1.96x  
C= 64, H=W= 28 | PT=0.019701 ms | CU=0.009531 ms | speedup=2.07x
```

Performans

Latency and Troughput

```
Pointwise PT latency: 0.0372 ms  
Pointwise CU latency: 0.0235 ms  
PT throughput: 26918 samples/s  
CU throughput: 42516 samples/s  
Speedup: 1.58x
```

```
C= 16, H=W=112 | PT=0.021380 ms | CU=0.010069 ms | PT tp=46772/s | CU tp=99315/s | speedup=2.12x  
C= 32, H=W= 56 | PT=0.021131 ms | CU=0.009964 ms | PT tp=47323/s | CU tp=100366/s | speedup=2.12x  
C= 64, H=W= 28 | PT=0.025985 ms | CU=0.009973 ms | PT tp=38484/s | CU tp=100273/s | speedup=2.61x
```

Appendix 4.2.6. Cross GPU Performance Comparison (A100, L4, T4)

These results show that FP16 consistently improves latency and throughput over FP32 across all GPUs, with A100 delivering the highest overall performance, L4 offering strong efficiency, and T4 showing the largest relative FP16 gains despite lower absolute throughput.

```

=== FINAL RESULTS ===
Engine      Latency (ms)    Throughput (FPS)
FP32        0.3714         2692.36
FP16        0.2992         3342.45

```

A100

```

=== FINAL RESULTS ===
Engine      Latency (ms)    Throughput (FPS)
FP32        0.2857         3500.44
FP16        0.2577         3879.74

```

L4

```

=== FINAL RESULTS ===
Engine      Latency (ms)    Throughput (FPS)
FP32        0.6553         1526.03
FP16        0.3447         2900.84

```

T4

```

PyTorch Latency: 22.01592445373535 ms
PyTorch FPS: 45.42166748897679
ONNX Runtime Latency: 2.53363037109375 ms
ONNX Runtime FPS: 394.6905639469056
TensorRT FP16 Latency: 0.34925031661987305 ms
TensorRT FP16 FPS: 2863.275858067176

```

A100

```

PyTorch Latency: 21.297357082366943 ms
PyTorch FPS: 46.954182912580535
ONNX Runtime Latency: 3.946469783782959 ms
ONNX Runtime FPS: 253.39101900874866
TensorRT FP16 Latency: 0.23839330673217773 ms
TensorRT FP16 FPS: 4194.748643356195

```

L4

```

PyTorch Latency: 24.031813144683838 ms
PyTorch FPS: 41.611508627313604
ONNX Runtime Latency: 3.9933128356933594 ms
ONNX Runtime FPS: 250.41864766058828
TensorRT FP16 Latency: 0.6218981742858887 ms
TensorRT FP16 FPS: 1607.9802793251113

```

T4

10. REFERENCES

- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient convolutional neural networks for mobile vision applications*. arXiv preprint arXiv:1704.04861. <https://arxiv.org/abs/1704.04861>
- NVIDIA Corporation. (2024). *CUDA C++ programming guide*. NVIDIA Developer. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA Corporation. (2024). *CUDA C++ best practices guide*. NVIDIA Developer. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- NVIDIA Corporation. (2024). *NVIDIA Nsight Systems user guide*. NVIDIA Developer. <https://developer.nvidia.com/nsight-systems>
- NVIDIA Corporation. (2024). *NVIDIA Nsight Compute user guide*. NVIDIA Developer. <https://developer.nvidia.com/nsight-compute>
- NVIDIA Corporation. (2024). *NVIDIA TensorRT developer guide*. NVIDIA Developer. <https://developer.nvidia.com/tensorrt>
- NVIDIA Corporation. (2024). *ONNX GraphSurgeon documentation*. NVIDIA Developer. <https://docs.nvidia.com/deeplearning/tensorrt/latest/api/onnx-graphsurgeon-api.html>
- ONNX Consortium. (2024). *Open Neural Network Exchange (ONNX) documentation*. <https://onnx.ai/>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems, 32. <https://pytorch.org/>
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). *Searching for activation functions*. arXiv preprint arXiv:1710.05941. <https://arxiv.org/abs/1710.05941>
- Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*. Proceedings of the 36th International Conference on Machine Learning (ICML). <https://arxiv.org/abs/1905.11946>
- TÜBİTAK ULAKBİM. (2024). *TRUBA user guide*. Türk Ulusal Bilim e-Altyapısı (TRUBA). <https://docs.truba.gov.tr>